Al Jazeera Media Network AI Proofreading Agent

AI-Powered Language & Editorial Review System

Technical Proposal

Advanced Business Computing W.L.L

CloudIT ME Technologies

ALJAZEERA

Reading Guide

This proposal is structured so each stakeholder can focus on the sections most relevant to their decision.

Reader	Recommended Sections
Executive / Sponsor	§1, §4, §8
Editorial Leadership / Managing Editors	§1, §2, §3.3, §5, §6
Language Reviewers	§2, §3, §5, Appendix E
Technology & Data Team	§3, §7, Appendices A/C/D
Legal / Compliance / Information Security	§5, §7, Appendix F

The main proposal is intentionally concise and decision-oriented. Detailed runtime schemas, infrastructure options, evaluation mechanics, and security controls are placed in appendices for specialist review.

1. Executive Summary and Proposal Positioning

1.1 Context and Objective

Al Jazeera operates in a high-pressure editorial environment where speed, linguistic precision, political sensitivity, sourcing discipline, neutrality, and consistency must work together without compromise. In this context, proofreading is not only a language-quality activity. It is an editorial control point that protects publishing integrity, institutional voice, audience trust, and alignment with Al Jazeera's internal standards.

The objective of this proposal is to define a practical, governed, and phased approach for building an AI Proofreading Agent that supports Al Jazeera editors in reviewing Arabic articles for language quality and style-guide compliance, while keeping final editorial authority fully with human editors.

The system is not intended to replace editors, approve publishing, rewrite articles freely, add unsupported facts, or make sensitive editorial decisions on behalf of Al Jazeera.

The guiding principle is:

> **AI suggests. Editors decide.** The system supports editorial judgment; it does not replace it.

1.2 Proposed Solution in Brief

We propose the AI Proofreading Agent as a secure, Google Cloud-native Arabic Editorial Reasoning System.

Unlike a generic proofreader, the proposed system is designed around the real structure of Al Jazeera's editorial challenge: Arabic language, entities, aliases, sourcing, quotes, captions, headlines, editorial voice, sensitive terminology, and evolving internal rules.

At runtime, the system will:

- receive an Arabic article from WordPress/CMS or a future integration channel;
- preserve the article as structured segments with zones and offsets;
- generate low-cost mechanical recommendations for known spelling, formatting, naming, and house-style issues;
- detect candidate editorial-rule triggers, including anchor-less relational risks such as loaded framing, implicit blame, attribution-strength shifts, indirect quotation, source/voice ambiguity, and headline/caption framing;

- retrieve relevant rule and entity context from Al Jazeera's approved knowledge base using article segments, mechanical recommendations, entity/alias matches, LLM-discovered rule-trigger candidates, and lightweight relationship expansion across directly linked entities/rules;
- assemble a compact judgment payload containing candidate spans, candidate entities, linked relationship context, retrieved rule cards, examples, exceptions, context zones, and routing signals;
- use controlled Gemini-based reasoning to apply relevant editorial rules — routed to a fast single-call path or a slower escalated path depending on retrieval confidence, sensitivity, ambiguity, and rule-trigger complexity;
- validate suggestions structurally and, where warranted, semantically before they are shown;
- present findings to editors with explanation, risk level, and accept/reject/edit controls;
- capture editor feedback to improve rule clarity, retrieval quality, evaluation, and governance.

For Phase 0, the relationship expansion can be implemented through a simple table-based structure, such as entity records with directly linked entities/rules and relationship labels. Based on Phase 0 findings, Phase 1 can either continue with this lightweight relationship model or upgrade to a native graph tool if the additional complexity is justified.

1.3 What Makes This Approach Different

This approach is different from a generic AI proofreading tool in five ways.

First, it treats Al Jazeera's rules as a living editorial knowledge base, not as static prompts or hardcoded checks. Rules, approved terms, entity records, examples, aliases, and temporal descriptors must be versioned, retrievable, auditable, and governed.

Second, it recognizes that many editorial rules are relational, not mechanical. The system must consider which entity is being described, where the phrase appears, whether it is in Al Jazeera's voice or a quote, and whether the wording carries a sensitive editorial frame.

Third, it separates recommendation, reasoning, validation, and approval. A machine-generated suggestion is not treated as safe simply because it sounds fluent.

Fourth, it avoids silent machine replacement. Even low-cost mechanical detection is treated as structured recommendation generation. Final handling remains governed by the workflow and editorial policy.

Fifth, it starts with a paid proof of concept. Because Al Jazeera has requested confidence that the approach works in practice, Phase 0 validates the method on controlled Al Jazeera-provided samples before committing to the full production MVP.

1.4 Delivery Strategy

The project should be delivered in controlled phases.

Phase	Purpose	Nature
Phase 0 — Discovery, Data Readiness, and Proof of Concept	Validate the approach on real Al Jazeera samples and produce a simplified working machine	Paid discovery / proof of concept
Phase 1 — Production MVP	Build the approved Arabic article proofreading and style-compliance workflow	Production MVP
Later Phases	Expand to additional integrations, content types, dashboards, languages, and workflows	Controlled expansion

Phase 0 is not a free pre-sales demo and not a production system. It is the first paid project phase, designed to reduce risk for both Al Jazeera and the delivery team.

1.5 Expected Value

Value Area	Expected Value
Editorial quality	More consistent detection of language, naming, sourcing, framing, and style-guide issues
Operational efficiency	Faster review of routine issues and clearer prioritization of high-risk findings
Governance and trust	Explainable findings, validation status, audit logs, editor feedback, and governed rule/entity updates

1.6 Recommended Next Step

The recommended next step is to proceed with Phase 0 — Discovery, Data Readiness, and Proof of Concept.

This phase allows Al Jazeera to validate the proposed approach on real sample material before investing in the full production build. It also allows the delivery team to confirm data readiness, benchmark initial performance, identify rule/entity structuring needs, and produce a precise Phase 1 implementation plan based on evidence.

2. Understanding Al Jazeera's Editorial Challenge

2.1 Arabic Proofreading Is Not Only Grammar

Al Jazeera's requirement is not limited to correcting spelling, grammar, punctuation, or formatting. These are important, but they are only one layer of the editorial challenge.

In Al Jazeera's environment, proofreading is also connected to editorial voice, approved terminology, entity naming, sourcing discipline, headline and caption sensitivity, quote handling, political and cultural context, and consistency with internal editorial standards.

Therefore, the AI Proofreading Agent must not behave like a generic grammar checker. It must distinguish between simple language recommendations and editorial-risk signals that require human judgment.

2.2 Mechanical Rules vs. Relational Rules

A key distinction in this proposal is between mechanical rules and relational rules.

Mechanical rules have a stable expected outcome. Examples include approved spellings, common name corrections, number/date formatting recommendations, punctuation conventions, and house-style forms. In this solution, the mechanical layer generates structured recommendations. It does not silently replace article text.

Relational rules depend on context. A word or phrase cannot be judged in isolation. The system may need to know:

- which entity is being described;
- whether the wording appears in a headline, caption, body, quote, or source attribution;
- whether it is Al Jazeera's own voice or someone else's claim;
- whether the entity has a sensitive editorial profile;
- whether the wording carries a prohibited or discouraged editorial meaning.

This distinction shapes the architecture. A typo, an approved spelling, a headline framing issue, and a sensitive descriptor cannot all be processed through the same simple correction logic.

2.3 Entities, Aliases, and Temporal Descriptors

Many editorial rules depend on the entity being discussed. A person, organization, movement, military body, political group, country, institution, or publication may appear under different names, aliases, abbreviations, transliterations, or descriptive references.

Before the system can apply the correct rule, it needs to retrieve and reason over possible entity matches. Al Jazeera may have an approved name, approved descriptor, sensitive policy profile, or temporal status for an entity.

Temporal descriptors matter because a public figure may be current, former, deceased, acting, resigned, removed, or appointed. The correct wording may depend on the status at the time of publication. This requires versioned and maintainable entity data, not static AI assumptions.

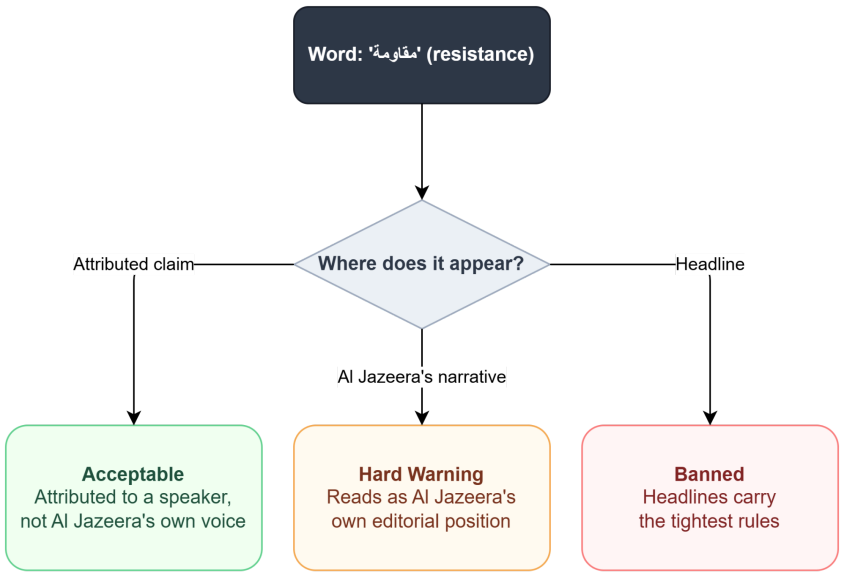
2.4 Quote, Source, Caption, Headline, and Al Jazeera Voice

The same phrase can carry different editorial meaning depending on where it appears.

A phrase inside a direct quote may be acceptable because it is attributed to a speaker. The same phrase in Al Jazeera's own narrative may be risky. A phrase in a headline may be more sensitive than the same phrase in a body paragraph. A caption may require stricter control because it often attaches a claim directly to an image or visual context.

The system must therefore distinguish between headline, subheadline, caption, article body, direct quote, indirect quote, source attribution, and Al Jazeera's own narrative voice.

Figure 2.4 — the same word, three verdicts, decided entirely by zone:



No keyword list can encode this — the same word needs three different verdicts depending on where it sits, which is why the architecture in §3 resolves zone before it judges wording.

2.5 Editorial Decision Spectrum

The system should support a decision spectrum, not only “correct” or “incorrect.”

Decision	Meaning
Acceptable	No issue under the active rule
Suggest	Acceptable but can be improved
Replace	Specific replacement is recommended
Soft Warning	Context-dependent issue requiring review
Hard Warning	Sensitive issue requiring explicit attention
Ban	Active prohibited usage or rule conflict

This decision taxonomy should shape the system output, editor interface, validation, feedback, and UAT approach.

2.6 What This Means for the Solution Design

The solution must be designed around editorial reasoning rather than generic correction. This means:

- the article must be segmented and preserved with stable IDs and offsets;
- mechanical recommendations should be generated cheaply but not silently applied;
- rules and entities must be retrieved from an approved, versioned knowledge base;
- Gemini should reason over a curated context packet, not model memory alone;
- validation must include both structural checks and semantic safety checks;
- sensitive outputs must fail safe to editor review;
- editor actions should become structured feedback for improvement.

---

3. Proposed Solution and Technical Approach

3.1 Solution Design Principles

The solution is based on four principles.

Principle	Meaning
AI suggests, editors decide	The system assists editors but does not replace editorial authority
Retrieve before judging	The system uses approved rule/entity knowledge instead of relying on model memory
Validate before showing	Suggestions are checked before being presented as usable recommendations
Fail safe to editor review	If the system is uncertain, it escalates instead of forcing a correction

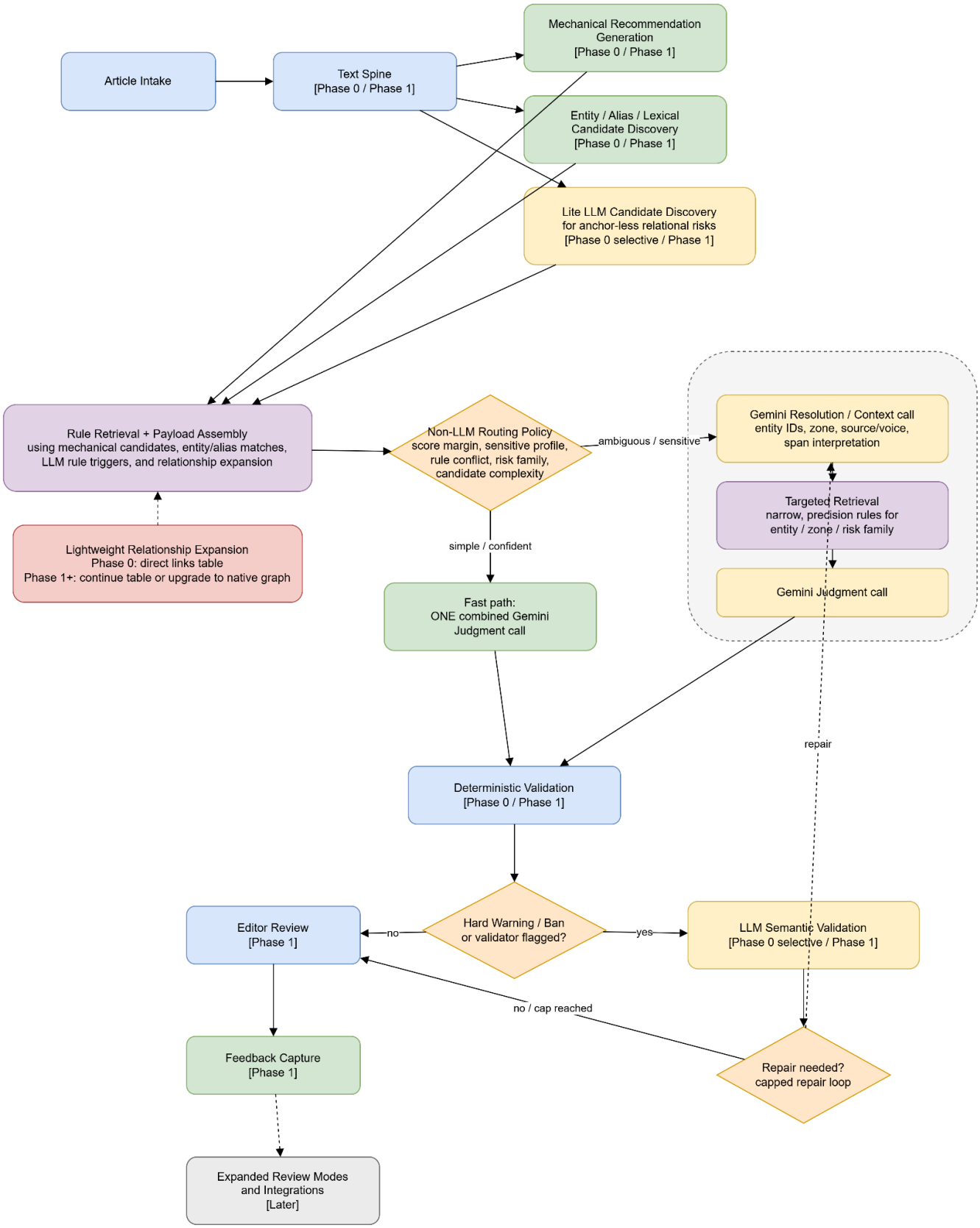
3.2 End-to-End Runtime Flow

The runtime flow does not rely on retrieval alone to discover every editorial risk. Some relational risks have no simple lexical, entity, or alias anchor. For example, loaded framing, implicit blame, attribution-strength shifts, indirect quotation, or Al Jazeera voice ambiguity may only become visible after reading the sentence or paragraph as discourse.

For this reason, the system includes a lightweight LLM candidate-discovery step before rule retrieval and routing. This step does not make final judgments and does not decide that a violation exists. It identifies candidate spans and risk families that may require editorial-rule retrieval.

The retrieval layer then uses all available signals — mechanical candidates, entity/alias matches, lexical/similarity matches, LLM-discovered rule-trigger candidates, and lightweight relationship expansion across directly linked entities/rules — to retrieve relevant mechanical and relational rule cards. Only after that enriched payload exists does the non-LLM router choose the execution path.

Figure 3.2 — the full runtime flow, including candidate discovery, relationship expansion, rule retrieval, and the fast/escalated branch.


Article Intake
Text Spine
[Phase 0 / Phase 1]
Mechanical Recommendation Generation
[Phase 0 / Phase 1]
Entity / Alias / Lexical Candidate Discovery
[Phase 0 / Phase 1]
Lite LLM Candidate Discovery for anchor-less relational risks
[Phase 0 selective / Phase 1]
Rule Retrieval + Payload Assembly
using mechanical candidates, entity/alias matches, LLM rule triggers, and relationship expansion
Lightweight Relationship Expansion
Phase 0: direct links table
Phase 1+: continue table or upgrade to native graph
Non-LLM Routing Policy
score margin, sensitive profile, rule conflict, risk family, candidate complexity
ambiguous / sensitive
simple / confident
Gemini Resolution / Context call
entity IDs, zone, source/voice, span interpretation
Targeted Retrieval
narrow, precision rules for entity / zone / risk family
Gemini Judgment call
Fast path:
ONE combined Gemini Judgment call
repair
Deterministic Validation
[Phase 0 / Phase 1]
Hard Warning / Ban or validator flagged?
no
yes
Editor Review
[Phase 1]
LLM Semantic Validation
[Phase 0 selective / Phase 1]
no / cap reached
Repair needed?
capped repair loop
Feedback Capture
[Phase 1]
Expanded Review Modes and Integrations
[Later]

3.2.1 [Phase 0 / Phase 1] Text Spine and Article Segmentation

The system first converts the article into a structured text spine that preserves article ID, article version, headline, captions, paragraphs, quote spans, source attribution zones where detectable, segment IDs, offsets, metadata, and original raw text.

This enables accurate highlighting, validation, auditability, and safe review.

3.2.2 [Phase 0 / Phase 1] Mechanical Recommendation Generation

The system generates structured recommendations for stable issues such as known spelling variants, approved house spellings, approved entity names, source names, country/city names, punctuation candidates, number/date format candidates, and spacing candidates.

This layer is a recommendation generator, not a replacement engine. It produces standardized candidate objects that can be confirmed, corrected, validated, or rejected later — for example, a candidate flagging "الجزيره" against the approved form "الجزيرة" (see Appendix A.3 for the full object schema).

This step can run in parallel with entity/alias discovery and the lite LLM candidate-discovery step. Together, these candidate signals become the inputs for rule retrieval and payload assembly.

3.2.3 [Phase 0 / Phase 1] Retrieval Pack Builder

Before the judgment model is called, the system must prepare the candidate universe that may trigger Al Jazeera rules.

This candidate discovery stage combines four signal sources:

1. **Mechanical candidates** — approved spelling, formatting, naming, and house-style recommendations.
2. **Entity / alias / lexical candidates** — known entities, aliases, approved names, sensitive profiles, semantic-field matches, and similarity-based matches.
3. **Lite LLM rule-trigger candidates** — possible anchor-less relational risks that may not be discoverable through regex, exact lookup, entity matching, or similarity search alone.
4. **Lightweight relationship expansion** — directly linked entities, rule families, policy profiles, or related rule cards retrieved from table-based links in Phase 0.

For Phase 0, this relationship expansion can be implemented without a native graph database. A simple table-based structure is sufficient. For example, an entity record can include a child table of linked entities/rules with two core fields:

Field	Purpose
relationship	Describes the relationship type, such as alias_of, related_to, governed_by_rule, sensitive_profile, historical_relation, successor/predecessor, or related_rule_family
linked_record	Points to the related entity, rule, profile, or rule family

During retrieval, when an entity or candidate risk is detected, the system expands one level across its direct links and adds this context to the judgment payload. This gives the model useful editorial neighborhood information without committing Phase 0 to a full graph platform.

The lite LLM step does not decide that the article has violated a rule. It identifies candidate spans and risk families that should be used for rule retrieval. Examples include:

- possible loaded framing;
- implicit blame framing;
- attribution-strength shift;
- unattributed claim;
- indirect quotation risk;
- source/voice ambiguity;
- headline framing risk;
- caption framing risk;
- motive or causality implied without sufficient attribution.

The retrieval layer then uses all candidate signals to retrieve relevant rule cards, entity records, aliases, approved names, examples, exceptions, semantic fields, sensitive policy profiles, temporal descriptors, linked relationship context, and previous editorial decisions where available.

The goal is to create a compact, relevant judgment payload for Gemini. The model should not infer Al Jazeera's policy from general training data, and it should not receive the entire rule repository. It should receive the rule cards and evidence most relevant to the candidate spans discovered in the article.

Based on Phase 0 findings, Phase 1 may either continue with the lightweight table-based relationship expansion model or upgrade to a native graph tool if the relationship complexity, retrieval quality, or governance needs justify it.

3.2.4 [Phase 1] Non-LLM Routing Policy

The router is a lightweight, versioned execution policy. It does not perform deep semantic reasoning and does not decide editorial judgment.

It uses observable signals from the assembled payload, specifically:

- **retrieval confidence / score margin per mention** — the single most important entity/retrieval ambiguity signal, since candidate count alone does not distinguish a genuinely ambiguous mention from overlapping high-confidence matches;
- presence of a known sensitive policy profile on any candidate entity;
- conflicting active rules on the same span;
- LLM-discovered risk family, such as implicit blame framing, unattributed claim, source/voice ambiguity, or headline/caption framing;
- whether the candidate span has an entity anchor or is anchor-less;
- article length and selected review mode;
- issue type and candidate complexity.

Its role is operational routing: choosing the fast single-call path or the escalated resolve-then-judge path, validation depth, and review mode. Because this policy directly determines whether high-risk content gets the deeper reasoning path, it is itself versioned and subject to the same regression gate as prompts and rules.

The router remains non-LLM. The lite LLM may generate structured candidate rule-trigger signals, but the routing decision itself is made by a deterministic, versioned policy.

3.2.5 [Phase 0 / Phase 1] Gemini Reasoning — Fast and Escalated Paths

Gemini reasoning begins only after candidate discovery and rule retrieval have produced an enriched judgment payload. The judgment model should not receive the whole rule repository, and it should not be asked to infer Al Jazeera policy from memory.

Fast path (single combined call): used when the router finds a simple, confident, low-sensitivity case. This may include mechanical-heavy findings, high-confidence entity matches, or low-risk rule applications. Gemini receives a structured judgment packet containing article segments, mechanical recommendations, retrieved rules, candidate entities, LLM-discovered candidate triggers where applicable, context zones, allowed/prohibited actions, output schema, and decision taxonomy. It returns a structured finding directly: acceptable, suggest, replace, soft warning, hard warning, ban, or needs editor review.

Escalated path (resolution, then targeted retrieval, then judgment): used when the router detects ambiguity, a sensitive entity, a rule conflict, or an anchor-less relational risk. A first Gemini call resolves the context more precisely: which entity is meant if any, which span is under review, whether the wording is in quote, narrative, headline, caption, source attribution, or Al Jazeera voice, and how the candidate risk family should be interpreted. The retrieval layer then pulls a narrower, precision-oriented

set of rule cards specific to the resolved entity, zone, and risk family. A second Gemini call performs the actual judgment against that narrowed, targeted packet.

Splitting candidate discovery, rule retrieval, resolution, and judgment is what makes entity-position-sensitive and anchor-less relational rules enforceable with precision. Collapsing all of this into one generic prompt may be acceptable for simple cases, but it is not safe enough for the cases that most need editorial care.

In both paths, the model must not invent rules, entities, facts, or policy. If context is insufficient, the output must indicate uncertainty or require editor review.

3.2.6 [Phase 0 / Phase 1] Deterministic Validation

Deterministic validation checks structural safety:

- schema validity;
- segment IDs;
- offsets;
- rule IDs;
- entity IDs;
- quote span integrity;
- patch applicability;
- unsupported patch operations.

This step answers: “Can this output be safely represented and reviewed against the original article?”

It is cheap, fully deterministic, and included from Phase 0 onward.

3.2.7 [Phase 0 Selective / Phase 1] LLM Semantic Validation and Repair Loop

For sensitive, relational, or high-risk outputs — specifically, any output that reached a Hard Warning/Ban decision or that the deterministic validator flagged — the system runs semantic validation to check whether a suggestion:

- preserves meaning;
- preserves attribution;
- avoids unsupported facts;
- avoids editorial stance drift;
- avoids new loaded terms;
- follows the minimum-change principle.

If semantic validation fails and the issue is repairable, the system loops back to the escalated Gemini judgment step with the failure reason. The repair loop is capped at one attempt on the fast path and two

attempts on the escalated path. If the system still cannot produce a validated safe suggestion, the finding is escalated to editor review — never silently approved and never silently discarded.

3.2.8 [Phase 1] Editor Review Output and Feedback Capture

The editor receives structured findings, not hidden rewrites. Each finding should include original text, suggested text where applicable, decision level, rule reference, entity reference where applicable, risk reason, validation status, and accept/reject/edit/escalate controls.

Every editor action becomes structured feedback: accepted, rejected, edited, escalated, ignored, corrected rule, corrected entity, or corrected severity.

3.2.9 [Later] Expanded Review Modes and Integrations

Later phases may extend the same reasoning core to Google Docs, additional CMS platforms, social media text, video/program scripts, investigations, dashboards, entity management, rule authoring UI, English, and multilingual support.

These are controlled expansions, not Phase 1 commitments.

3.3 Editor Experience and Worked Example

The editor should not see a black-box AI response. The editor should see highlighted findings in the article, classified by severity and type.

Each finding should show:

- highlighted phrase;
- suggested action;
- reason;
- rule or entity reference where available;
- validation status;
- available actions.

Suggestion types should include mechanical recommendation, format recommendation, style suggestion, replacement recommendation, soft warning, hard warning, banned usage, needs editor review, and validation failed.

A proof object should explain important findings using structured fields such as rule ID, entity ID, segment type, article zone, detected risk, retrieved example, and validation result.

Worked example: anchor-less relational risk

An article sentence frames an unnamed group's action using wording that may imply blame or loaded causality, but the sentence does not contain a clear banned word, known entity name, or obvious alias.

The text spine preserves the sentence as a segment. The mechanical layer does not flag it because it is not a spelling or formatting issue. Entity/alias retrieval may also return no strong match because the actor is unnamed.

The lite LLM candidate-discovery step identifies the span as a possible rule trigger, for example:

```
{
 "candidate_type": "possible_rule_trigger",
 "risk_family": "implicit_blame_framing",
 "segment_id": "S014",
 "span": "...",
 "requires_rule_retrieval": true
}
```

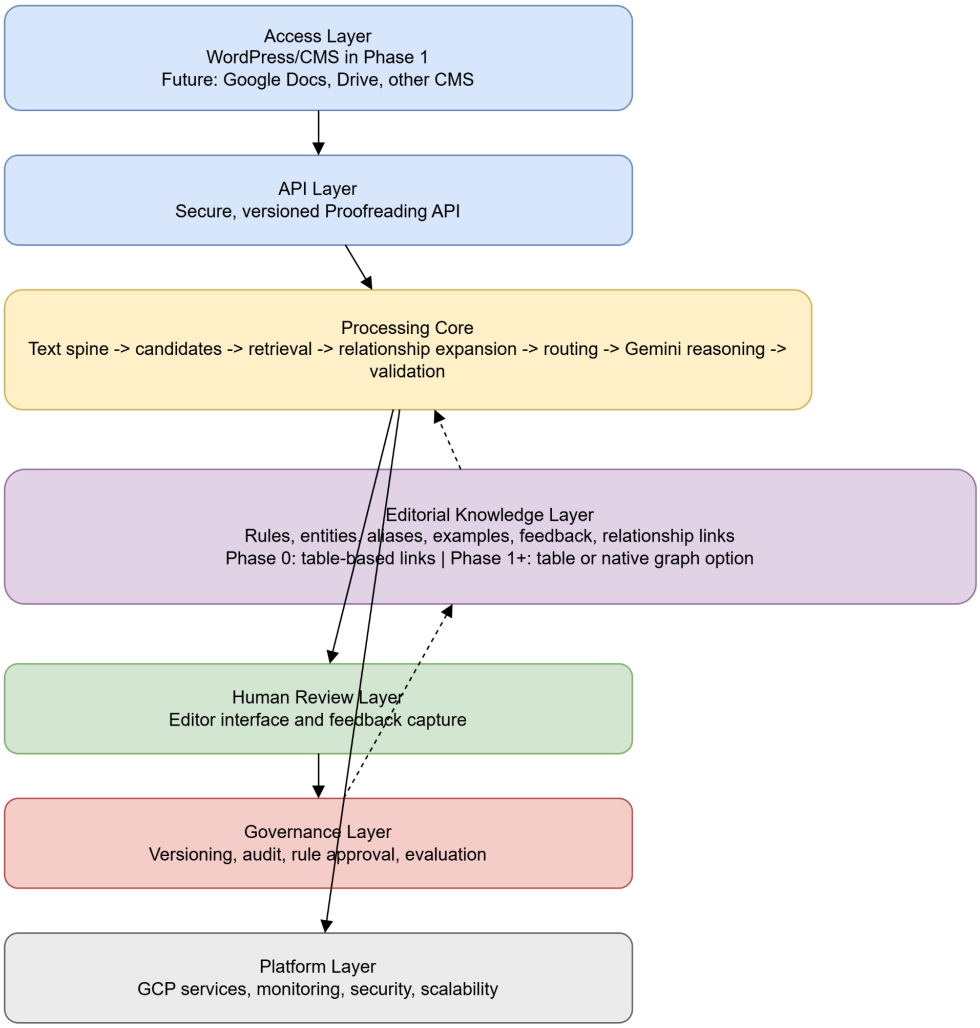
The retrieval layer uses this candidate risk family to retrieve relevant rule cards about blame attribution, loaded causality, source attribution, and Al Jazeera voice. The router sees an anchor-less relational risk and routes the case to the escalated path.

In the escalated path, Gemini first resolves the context: whether the sentence is Al Jazeera's own voice or attributed to a source, whether the causality is stated or implied, and whether the wording requires editorial review. Targeted retrieval then narrows the rule cards to the exact risk family and context zone. The judgment call applies those rules and may return a soft warning, hard warning, or needs editor review.

If a safe minimal alternative exists, it is shown to the editor. If the system cannot produce a validated safe suggestion, it displays the warning without forcing a rewrite. The editor decides whether to edit, keep, replace, or escalate.

3.4 Architecture Summary

Figure 3.4 — logical architecture layers:



The logical architecture consists of:

Layer	**Purpose**
Access layer	WordPress/CMS initially, future channels later
API layer	Secure, versioned proofreading API
Processing core	Text spine, recommendations, candidate discovery, retrieval, relationship expansion, routing, judgment, validation
Editorial knowledge layer	Rules, entities, aliases, examples, feedback, and linked relationship context
Human review layer	Editor interface and feedback capture

Governance layer	Versioning, audit, rule approval, evaluation
Platform layer	GCP services, monitoring, security, scalability

The proposed GCP deployment may use Vertex AI/Gemini, vector retrieval services, Cloud Run or GKE, Cloud Storage, BigQuery, IAM, KMS, Secret Manager, Cloud Logging, Cloud Monitoring, and Cloud Audit Logs. Final service selection should be confirmed during Phase 0 and early Phase 1 based on Al Jazeera's standards.

Phase 1 focuses on WordPress/CMS integration through a versioned API so that later integrations can reuse the same reasoning core.

The editorial knowledge layer will support lightweight relationship expansion from Phase 0. This can initially be implemented with table-based links between entities, rules, profiles, examples, and related records. Based on Phase 0 findings, the project may continue with this approach or upgrade to a native graph tool in Phase 1 or later if it improves retrieval quality, maintainability, or governance.

4. Scope and Phased Delivery

4.1 Scope Principles

The project should be delivered through controlled phases: build from validated evidence; commit only to approved scope; expand through agreed phase gates. This prevents the proposal from becoming a broad promise to solve every editorial automation scenario before the core approach is proven.

4.2 Phase 0 — Discovery, Data Readiness, and Proof of Concept

Phase 0 is a paid discovery and proof-of-concept phase. Its purpose is to validate the proposed approach on real Al Jazeera sample material and produce a precise Phase 1 plan.

Area	Phase 0 Scope
Objectives	Validate approach, test simplified machine, understand data readiness, structure limited rule/entity subset, test lightweight relationship expansion, measure initial quality/latency, and test anchor-less relational candidate discovery
Inputs from Al Jazeera	Representative Arabic articles, selected mechanical and relational rules, examples, limited entity list, relationship examples between entities/rules where available, expected outputs where available, workflow access for discovery

Activities	Workshops, data review, limited rule/entity structuring, lightweight table-based relationship model, text spine setup, mechanical recommendations, entity/alias discovery, lite LLM candidate discovery for relational rule triggers, rule retrieval using all candidate signals, relationship expansion into the payload, Gemini judgment, deterministic validation, semantic validation on selected high-risk examples, review sessions, initial benchmark
Deliverables	Discovery findings, data readiness assessment, limited rule/entity repository, lightweight relationship expansion model, simplified working machine, sample outputs, initial benchmark, feedback summary, Phase 1 plan
Not included	Production deployment, complete rule/entity coverage, final latency guarantees, full WordPress integration, native graph platform unless explicitly agreed, dashboards, multilingual support, non-article content types

Phase 0 acceptance should be based on demonstration of the simplified working machine, structured findings, limited retrieval, candidate rule-trigger discovery, judgment/validation flow, editor-readable outputs, initial benchmark results, and a Phase 1 implementation plan.

4.3 Phase 1 — Production MVP

Phase 1 builds the production MVP for the approved Arabic article scope confirmed during Phase 0.

Area	Phase 1 Scope
Functional deliverables	Arabic article intake, text spine, mechanical recommendations, entity/alias candidate discovery, lite LLM candidate discovery for anchor-less relational risks, rule retrieval and payload assembly, relationship expansion using the Phase 0 model or an approved upgraded graph approach, routing policy, Gemini reasoning (fast and escalated paths), deterministic validation, semantic validation where required, capped repair loop, editor review, feedback capture
Integration deliverables	WordPress/CMS submission flow, versioned API, result rendering, storage of review outputs, audit/event logging, testing/UAT environment
Governance deliverables	Rule/entity versioning, relationship-link governance, review snapshot policy, feedback store, regression testing process, model/prompt/rule/routing change control

UAT	Mechanical accuracy, relational rule handling, candidate trigger quality, relationship expansion usefulness, editor acceptance, latency by mode, validation reporting, auditability, workflow acceptance, access control

Unless explicitly added, Phase 1 does not include full multilingual support, Google Docs integration, social media workflow, video/program script review, full dashboards, full self-service rule management UI, Asana automation, production fact-checking, or link review.

4.4 Later Phases — Controlled Expansion

Later phases may include: Google Docs and broader CMS integrations; Google Drive and workflow automation; dashboards and analytics; richer rule/entity management UI; social media and video/program script review; long-form and investigations support; English and multilingual support; advanced escalation and reviewer assignment.

These are expansion options, not Phase 1 commitments.

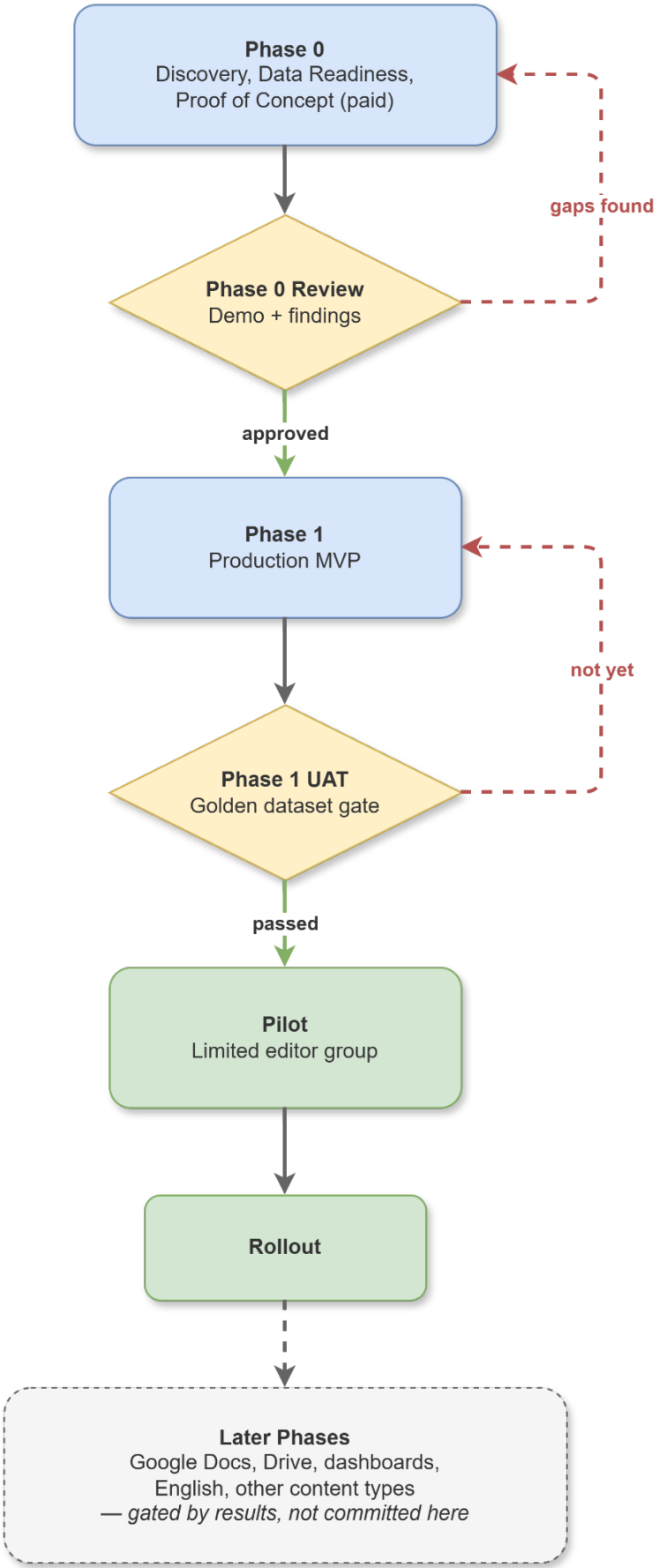
4.5 Out of Scope

The system will not approve publication, make autonomous editorial decisions, add unsupported facts, change editorial stance, replace the editorial hierarchy, perform full fact-checking in Phase 1, verify links in Phase 1, publish automatically, or train on unpublished content without approval.

4.6 Phase Gates and Decision Points

Gate	Decision
Phase 0 kickoff	Project allocation, payment placement, sample data access
Phase 0 review	Demonstration and findings
Phase 1 approval	Confirmed scope, UAT criteria, timeline, dependencies
Phase 1 UAT	Acceptance against agreed dataset/workflow
Pilot / rollout	Controlled adoption by selected users
Later phase planning	Based on results, priorities, and readiness

Figure 4.6 — the phase and gate sequence:


Phase 0
Discovery, Data Readiness,
Proof of Concept (paid)
gaps found
Phase 0 Review
Demo + findings
approved
Phase 1
Production MVP
not yet
Phase 1 UAT
Golden dataset gate
passed
Pilot
Limited editor group
Rollout
Later Phases
Google Docs, Drive, dashboards,
English, other content types
— gated by results, not committed here

5. Editorial Governance, Responsible AI, and Human Control

5.1 Human-in-the-Loop Principles

The AI Proofreading Agent is assistive. It may detect candidate issues, recommend wording, classify risk, and explain findings. Editors remain responsible for accepting, rejecting, editing, or escalating suggestions.

The system must not publish, approve, add facts, change editorial stance, or bypass Al Jazeera's editorial hierarchy.

5.2 Allowed, Reviewable, and Prohibited Actions

Category	Examples
Allowed recommendations	Spelling, punctuation, formatting, terminology consistency, clarity, source/quote/voice warnings
Reviewable actions	Relational findings, sensitive descriptors, headline/caption framing, attribution changes, hard warnings, banned usage flags
Prohibited actions	Publishing, adding facts, deleting/adding paragraphs, changing editorial angle, changing conflict-party descriptions without editor decision, removing attribution in a meaning-changing way, silently replacing sensitive wording

These limits should be enforced through output schemas, validation, UI controls, and audit logs.

A note on automation level: Al Jazeera's own requirements permit fully automated handling of mechanical corrections such as spelling, punctuation, and date/number formats with no per-instance editor sign-off. This proposal recommends routing mechanical recommendations through a lightweight review step in Phase 1 as well — shown to the editor as low-friction suggestions rather than silent edits — so that trust in the system is established before any auto-apply behavior is turned on. We recommend introducing a configurable auto-apply mode for the mechanical layer once Phase 1 acceptance data shows it is warranted, rather than defaulting to it from day one. This is a deliberate, more conservative choice than the original requirement allows, and is called out here so it is a decision Al Jazeera signs off on rather than an unstated deviation.

5.3 Rule and Entity Governance

The solution depends on a governed editorial knowledge base containing style-guide rules, mechanical recommendations, approved names, entity records, aliases, sensitive policy profiles, temporal descriptors, examples, exceptions, and historical editorial decisions where available.

Rules and entities must be versioned and auditable. The AI model must not invent Al Jazeera policy. If a rule, entity, or approved descriptor is missing or ambiguous, the system should flag uncertainty rather than create its own policy.

5.4 Rule Authoring Governance Pipeline

Rule updates follow a controlled, human-approved process: an authorized reviewer proposes a change, the system surfaces related rules/entities/examples and any duplication or conflict, and the change only becomes active after structured validation, testing against examples, and reviewer approval. The AI may assist drafting and analysis, but it never activates a rule directly.

The full workflow, including the LLM-assisted analysis step and the audit record produced at activation, is specified in Appendix E.4 rather than repeated here.

5.5 Versioning, Audit, and Rule Snapshots

Each review should be linked to article version, rule version, entity version, model version, prompt/schema version, routing policy version, validation version, timestamp, and editor actions.

If a rule changes after an article review begins, the review should remain linked to the rule snapshot active at submission unless Al Jazeera defines a different reprocessing policy.

5.6 Responsible AI Guardrails

Responsible AI must be embedded inside the workflow. Guardrails should cover meaning preservation, attribution preservation, stance drift prevention, no unsupported facts, no hidden rewriting, no autonomous publishing, sensitive terminology handling, explainability, auditability, privacy, and security.

Important findings should include proof-style explanations based on structured fields such as rule ID, entity ID, context zone, retrieved evidence, and validation status.

5.7 Fail-Safe Behavior

When uncertain, the system should fail safe: no silent correction; no invented rule or entity; no forced rewrite; no automatic approval; no downgrade of sensitive findings. If a safe suggestion cannot be validated, the system should return needs_editor_review.

---

6. Evaluation, Benchmarking, and Success Criteria

6.1 Phase 0 Feasibility Benchmark

Phase 0 should include an initial feasibility benchmark using a controlled set of Al Jazeera-provided Arabic articles, selected rules, and selected entity examples.

It should measure whether the system can:

- ingest and segment articles;
- generate mechanical recommendations;
- detect candidate entities and aliases;
- identify possible anchor-less relational rule triggers using the lite LLM candidate-discovery step;
- retrieve relevant rules using mechanical, entity, lexical, and LLM-discovered candidate signals;
- produce useful structured judgments;
- validate outputs deterministically;
- run semantic validation on selected high-risk examples;
- present editor-readable results;
- provide initial latency observations by stage.

This benchmark is a feasibility and calibration exercise, not a final production acceptance test

6.2 Initial Benchmark vs. Final Calibration

The initial Phase 0 benchmark may use a small sample, such as 20–30 representative articles, to validate feasibility and identify gaps. This is not enough for final production claims about p95 latency, full relational accuracy, final router calibration, or production acceptance rates.

The Phase 0 benchmark should specifically measure how often Al Jazeera sample articles trigger anchor-less relational risk detection, because this affects both architecture and latency planning.

Benchmark Type	Purpose
Phase 0 benchmark	Feasibility, first observations, candidate trigger behavior, and initial latency
Calibration benchmark	Threshold tuning, retrieval tuning, router calibration, and evaluation method refinement
Phase 1 UAT	Formal acceptance
Regression tests	Future rule/model/prompt/retrieval/routing release control

6.3 Golden Dataset Approach

A golden dataset should be created for UAT and regression testing. It should include Arabic articles, mechanical issues, entity/naming cases, relational rule examples, headline/caption cases, quote/source attribution cases, sensitive wording examples, and expected decisions or acceptable decision ranges.

Mechanical rules can often be evaluated with exact expected outputs. Relational rules may require adjudicated labels, acceptable severity bands, or multiple accepted outcomes.

6.4 Mechanical Rule Evaluation

Mechanical recommendations should be evaluated using detection recall, precision, recommendation accuracy, offset accuracy, duplicate detection, false positive rate, and consistency across reruns.

6.5 Relational Rule Evaluation

Relational rules should be evaluated using:

- correct entity resolution;
- context-zone detection;
- correct rule-trigger discovery;
- correct rule retrieval;
- correct rule activation;
- severity classification quality;
- suggestion safety;
- attribution preservation;
- stance preservation;
- editor acceptance rate;
- false positive severity;

- missed high-risk issue rate;
- routing accuracy, meaning whether the router sent the case down the path it actually needed.

For sensitive rules, missed high-risk issues may be more serious than ordinary false positives. The evaluation should therefore separately track missed anchor-less relational risks, such as implicit blame framing, unattributed claims, attribution-strength shifts, source/voice ambiguity, and headline/caption framing.

6.6 Latency Measurement

Latency should be measured by review mode, article length, content type, and risk category.

Relevant measurements include:

- text spine generation;
- mechanical recommendation generation;
- entity/alias/lexical candidate discovery;
- lite LLM candidate discovery;
- rule retrieval and payload assembly;
- routing;
- fast-path judgment;
- escalated-path resolution / targeted retrieval / judgment;
- deterministic validation;
- semantic validation;
- repair-loop time;
- total review time.

Exact latency targets should be confirmed during Phase 0 and finalized before Phase 1 UAT using real article sizes, selected GCP region, selected model tier, and agreed integration surface.

The proposal should not assume that all breaking-news content will behave like low-risk mechanical content. For Al Jazeera, breaking news may frequently involve conflict, political sensitivity, or attribution risk. Phase 0 should therefore measure the actual escalation rate rather than assuming it.

6.7 Editor Acceptance Metrics

The system should track accepted, rejected, edited, escalated, ignored, validation-failed, wrong-rule, wrong-entity, and useful-but-badly-worded findings.

Acceptance should not be interpreted naively. A rejected suggestion may indicate a system issue, or it may mean the editor chose a different valid wording.

6.8 Regression Testing for Future Changes

Regression testing is required for changes to:

- rules;
- entity data;
- prompts;
- candidate-discovery logic;
- retrieval configuration;
- routing policy;
- validation logic;
- model versions.

No major change should be promoted without testing against the golden dataset and selected adversarial cases.

Regression testing should specifically verify that anchor-less relational risks are not silently lost when prompts, retrieval configuration, routing thresholds, or model versions change.

---

7. Security, Operations, Roles, and Dependencies

7.1 Security and Privacy Principles

The AI Proofreading Agent will process unpublished editorial content. The solution should ensure that unpublished content remains within the approved environment, data is encrypted, access is role-based, actions are logged, retention policies are defined, secrets are managed, and no unauthorized third-party sharing or model training occurs.

7.2 GCP Security Controls

The proposed GCP deployment may use IAM, least-privilege service accounts, Cloud KMS, Secret Manager, Cloud Audit Logs, Cloud Logging, Cloud Monitoring, VPC/network restrictions where required, Cloud Storage and BigQuery retention policies, and separate development/staging/production environments.

Final configuration should follow Al Jazeera's approved cloud security standards.

7.3 Monitoring and Support Model

Monitoring should cover API availability, model call failures, retrieval failures, validation failures, latency by mode, processing errors, integration errors, and storage/logging health.

Editorial monitoring should cover high-risk issue counts, acceptance/rejection trends, frequent false positives, missed cases identified by editors, rule performance, validation-failed suggestions, and escalation patterns.

Support should include a defined process for incident handling, bug reporting, rule issues, and operational escalation.

7.4 Roles and Responsibilities Matrix

Role	Main Responsibilities
Al Jazeera Editorial Leadership	Approves governance model, confirms phase scope, owns final editorial decisions
Language Reviewers / Editorial Standards	Own rule quality, approve rule changes, help build golden dataset, review sensitive behavior
Technology and Data Team	Own environment standards, security review, integration readiness, operational handover
Editors / Pilot Users	Use the tool, review suggestions, provide feedback
Delivery Team	Builds solution, structures initial data with Al Jazeera input, implements validation/integration/testing/documentation
Joint Project Team	Reviews Phase 0 findings, confirms Phase 1 scope, manages risks, validates UAT readiness

7.5 Assumptions

The proposal assumes Al Jazeera will provide representative Arabic article samples, selected rules and examples, a limited entity list or validation support, editorial stakeholder availability, WordPress/CMS access for Phase 1, GCP acceptance unless otherwise specified, and timely confirmation of UAT criteria after Phase 0.

7.6 Client Dependencies

Key dependencies include article samples, rule documents, entity data, Language Reviewer availability, integration discovery access, WordPress/CMS technical information, cloud/security requirements, Phase 0 feedback, and UAT participation.

Delays in these dependencies may affect Phase 0 or Phase 1 timelines.

7.7 Key Risks and Mitigations

Risk	Mitigation
Rule/entity data not ready	Use Phase 0 to assess and structure a limited approved subset
Relational ambiguity	Use adjudicated examples, severity bands, and editor review
Anchor-less relational risks are missed by retrieval alone	Add lite LLM candidate discovery before rule retrieval; use discovered risk families to retrieve relevant rules; measure trigger quality in Phase 0
Latency higher than desired	Separate fast/escalated paths, run candidate discovery efficiently, and benchmark stage-level latency during Phase 0
Breaking news triggers escalation more often than expected	Measure actual escalation rate during Phase 0 rather than assuming breaking news is mostly low-risk
Golden dataset becomes outdated	Assign ownership and recertify periodically
Model behavior changes	Pin versions where possible and regression-test changes
Editors do not trust suggestions	Provide proof objects, validation status, and clear controls
Scope expands too early	Keep Phase 1 tied to Phase 0-confirmed scope

8. Conclusion and Recommended Next Step

8.1 Summary of the Proposed Approach

The proposed AI Proofreading Agent is a governed Arabic editorial reasoning system for Al Jazeera. It supports editors in reviewing Arabic articles for language quality, style-guide compliance, approved terminology, entity consistency, source/quote awareness, and sensitive editorial wording.

The system preserves the article, retrieves approved editorial context, applies controlled Gemini reasoning through a fast or escalated path depending on risk, validates outputs, presents explainable findings, and keeps final decisions with Al Jazeera editors.

8.2 Why Phase 0 Is the Right Next Step

Phase 0 answers the central question before full implementation:

> Can the proposed approach produce useful, safe, explainable editorial findings on real Al Jazeera Arabic article samples?

This requires a controlled working machine, selected rules, selected entity examples, sample articles, validation logic, and editor feedback. It cannot be proven by a slide deck alone.

Phase 0 protects both sides: Al Jazeera gets evidence before full investment, and the delivery team avoids guessing about data readiness, rule complexity, latency, and evaluation.

8.3 Immediate Next Actions

- Approve the proposal direction and Phase 0 delivery model;
- confirm Phase 0 commercial allocation and payment placement;
- nominate Al Jazeera editorial and technical stakeholders;
- provide representative Arabic article samples and selected rules/entities;
- run the Phase 0 discovery and proof-of-concept work;
- review Phase 0 findings and agree the final Phase 1 production MVP scope.

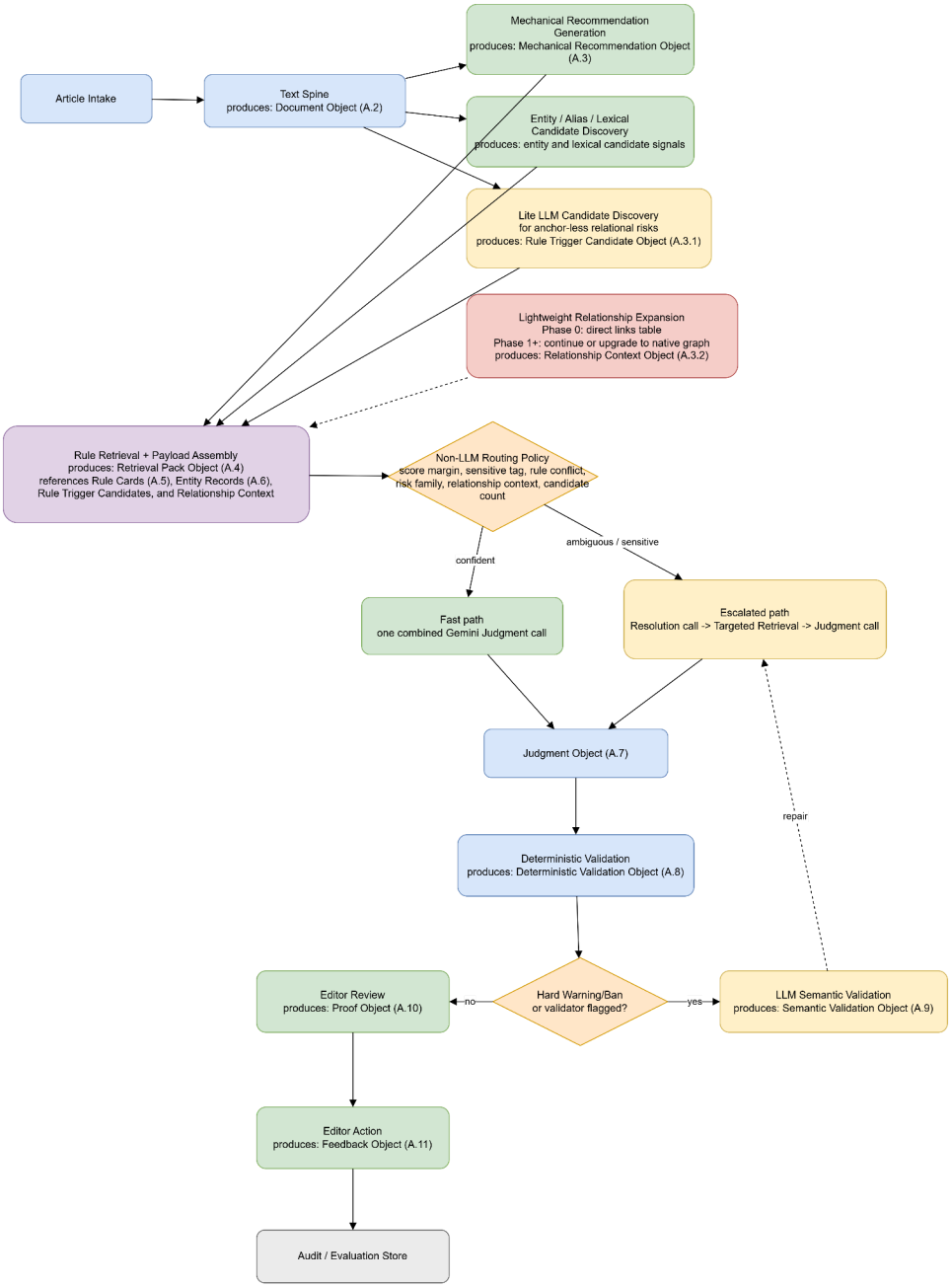
This approach gives Al Jazeera a practical path to validate the AI Proofreading Agent before full

production investment, while preserving editorial control, technical safety, and delivery discipline.

Appendix A — Detailed Runtime Pipeline and Schemas

A.1 Full Runtime Pipeline

Figure A.1 — the complete pipeline, annotated with the object each step produces:



The pipeline is designed to separate detection, retrieval, relationship expansion, reasoning, validation, and human decision-making. Every major step produces a structured, versioned object rather than relying on free-form model messages.

A.2 Text Spine Object

The text spine preserves the article structure and enables safe highlighting, validation, and audit.

```
{
 "document_id": "DOC-001",
 "article_version": "v1",
 "language": "ar",
 "source_system": "wordpress",
 "segments": [
  {
   "segment_id": "S001",
   "type": "headline",
   "text": "...",
   "start_offset": 0,
   "end_offset": 72
  }
 ],
 "metadata": {
  "section": "news",
  "submitted_by": "editor_id",
  "submitted_at": "timestamp"
 }
}
```

A.3 Mechanical Recommendation Object

Mechanical recommendations are generated cheaply but are not directly applied to the article. This is the canonical schema referenced from §3.2.2.

```
{
 "candidate_id": "MC-001",
 "candidate_type": "mechanical_recommendation",
 "segment_id": "S003",
 "original": "الجزيره",
 "recommended": "الجزيرة",
 "rule_id": "M-004",
 "source": "approved_form_index",
 "risk_level": "low",
 "requires_review": true
}
```


A.3.1 Rule Trigger Candidate Object

The Rule Trigger Candidate Object is produced by the lite LLM candidate-discovery step. It identifies candidate spans and risk families that may require rule retrieval. It is not a final judgment and does not claim that a violation has occurred.


```
{
 "trigger_id": "RTC-001",
 "candidate_type": "possible_rule_trigger",
 "segment_id": "S014",
 "span": "...",
 "risk_family": "implicit_blame_framing",
 "anchor_type": "anchorless",
 "requires_rule_retrieval": true,
 "retrieval_query_terms": [
  "implicit blame",
  "loaded causality",
  "unsupported attribution"
 ],
 "notes": "Candidate risk only; requires rule retrieval and judgment."
}
```


Typical risk_family values may include:

- loaded_framing;
- implicit_blame_framing;
- unattributed_claim;
- attribution_strength_shift;
- indirect_quotation_risk;
- source_voice_ambiguity;
- headline_framing;

- caption_framing;
- unsupported_motive_or_causality.

A.3.2 Relationship Context Object

The Relationship Context Object is produced by the lightweight relationship expansion layer. In Phase 0, this may be implemented as table-based direct links rather than a native graph. The goal is to add relevant editorial neighborhood context to the retrieval payload.


```
{
 "relationship_context_id": "RC-001",
 "source_record_id": "E-001",
 "source_record_type": "entity",
 "expanded_links": [
  {
   "relationship": "governed_by_rule",
   "linked_record_id": "R-017",
   "linked_record_type": "rule",
   "reason": "Entity has an active sensitive descriptor rule"
  },
  {
   "relationship": "related_rule_family",
   "linked_record_id": "RF-004",
   "linked_record_type": "rule_family",
   "reason": "Entity belongs to a sensitive conflict-party profile"
  }
 ],
 "expansion_depth": 1,
 "implementation_mode": "table_based_direct_links"
}
```


For Phase 0, the system should normally use direct one-hop expansion to avoid noisy context. If Phase 0 shows that deeper relationships are useful and manageable, Phase 1 may extend the expansion logic or upgrade to a native graph tool.

A.4 Retrieval Pack Object

The retrieval pack gives Gemini a controlled context instead of asking it to infer policy from memory. It is assembled using mechanical recommendations, entity/alias candidates, lexical/similarity signals, rule-trigger candidates from the lite LLM step, and relationship context from the lightweight expansion layer.


```
{
 "retrieval_pack_id": "RP-001",
 "document_id": "DOC-001",
 "mechanical_candidates": [],
 "candidate_entities": [],
 "rule_trigger_candidates": [],
 "relationship_context": [],
 "candidate_rules": [],
 "semantic_fields": [],
 "sensitive_profiles": [],
 "examples": [],
 "retrieval_notes": {
  "recall_priority": true,
  "ambiguous_matches": [],
  "confidence_margin_by_mention": {},
  "anchorless_risk_families": [
   "implicit_blame_framing"
  ],
  "relationship_expansion_mode": "table_based_direct_links"
 }
}
```


A.5 Rule Card Object

Rule cards represent active editorial rules in a structured, versioned format.

```
{
 "rule_id": "R-017",
 "version": "1.2",
 "title": "Sensitive descriptor usage",
 "rule_type": "relational",
 "decision_range": ["soft_warning", "hard_warning", "ban"],
 "applies_to": ["headline", "caption", "body"],
 "conditions": [],
 "examples": [],
 "exceptions": [],
 "effective_from": "date",
 "approved_by": "language_reviewer_id"
}
```


A.6 Entity Record Object

Entity records store approved names, aliases, status, and sensitive editorial profiles.


```
{
 "entity_id": "E-001",
 "entity_type": "organization",
 "approved_name_ar": "...",
 "aliases": [],
 "sensitive_policy_profile": true,
 "temporal_status": {
  "status": "active",
  "effective_date": "date"
 },
 "related_rules": ["R-017"],
 "version": "1.0"
}
```


A.7 Judgment Object

The judgment object is the structured output of the Gemini judgment step, whichever path produced it.

```
{
  "judgment_id": "J-001",
  "candidate_id": "MC-001",
  "decision": "replace",
  "segment_id": "S003",
  "rule_ids": ["M-004"],
  "entity_id": null,
  "path_taken": "fast",
  "suggested_text": "الجزيرة",
  "requires_editor_approval": true,
  "risk_flags": [],
  "reason_code": "approved_house_spelling"
}
```


A.8 Deterministic Validation Object

Deterministic validation checks whether the output is structurally safe.


```
{
  "validation_id": "DV-001",
  "judgment_id": "J-001",
  "status": "pass",
  "checks": {
    "schema_valid": true,
    "segment_exists": true,
    "offsets_valid": true,
    "rule_ids_valid": true,
    "entity_ids_valid": true,
    "quote_integrity_preserved": true,
    "patch_applicable": true
  }
}
```


A.9 Semantic Validation Object

Semantic validation checks whether meaning, attribution, and editorial stance are preserved.

```
{
 "semantic_validation_id": "SV-001",
 "judgment_id": "J-001",
 "status": "pass",
 "checks": {
  "meaning_preserved": true,
  "attribution_preserved": true,
  "no_new_facts": true,
  "no_stance_drift": true,
  "minimum_change": true
 },
 "repair_instruction": null
}
```


A.10 Proof Object

The proof object is what allows an editor or auditor to understand why a finding exists.


```
{
 "proof_id": "P-001",
 "judgment_id": "J-001",
 "rule_id": "R-017",
 "entity_id": "E-001",
 "segment_type": "headline",
 "risk_flags": ["sensitive_descriptor"],
 "validation_status": "passed",
 "editor_explanation": "The phrase touches an active sensitive descriptor rule in a headline context."
}
```


A.11 Feedback Object

Every editor action becomes structured feedback.

```
{
  "feedback_id": "F-001",
  "judgment_id": "J-001",
  "editor_action": "edited",
  "final_text": "...",
  "feedback_reason": "Suggestion useful but wording adjusted",
  "submitted_by": "editor_id",
  "submitted_at": "timestamp"
}
```

Appendix B — Phase Deliverables and Acceptance Matrix

B.1 Phase 0 Deliverables

Deliverable	Description
Discovery findings	Summary of editorial, data, and technical findings
Data readiness assessment	Assessment of rule/entity/article readiness
Limited rule/entity repository	Structured subset for proof of concept
Simplified working machine	Demonstrates the core runtime approach
Sample review outputs	Outputs on agreed Arabic article samples
Initial benchmark report	First observations on quality, latency, and gaps
Editorial feedback summary	Feedback from Al Jazeera reviewers
Phase 1 plan	Confirmed MVP scope, risks, dependencies, and implementation plan

B.2 Phase 0 Acceptance Criteria

Phase 0 is accepted when:

- agreed Arabic samples are processed;
- text spine generation is demonstrated;
- selected mechanical recommendations are generated;
- limited rule/entity retrieval is demonstrated;
- Gemini judgment produces structured findings on both the fast path and at least one escalated case;
- deterministic validation is demonstrated;
- semantic validation is demonstrated on selected high-risk examples;
- editor-readable results are reviewed;
- initial benchmark findings are documented;
- Phase 1 scope and plan are produced.

Phase 0 does not require full production performance or full rule coverage.

B.3 Phase 1 Deliverables

Deliverable	Description
Production runtime workflow	Article intake, recommendation, retrieval, routing, judgment, validation, review
WordPress/CMS integration	Submit article, receive findings, review suggestions
Rule/entity repository	Approved production subset confirmed after Phase 0
Validation layer	Deterministic and semantic validation where required
Editor review interface	Accept, reject, edit, escalate, and review explanations
Feedback capture	Structured editor feedback and correction records
Audit trail	Rule/model/version/action traceability
UAT package	Golden dataset, test results, and acceptance report

B.4 Phase 1 Acceptance Criteria

Phase 1 acceptance should be based on:

- agreed functional scope delivered;

- WordPress/CMS workflow accepted;
- mechanical recommendations meeting agreed quality threshold;
- relational findings meeting agreed evaluation method;
- fast and escalated path routing behaving as calibrated;
- semantic validation and fail-safe behavior demonstrated;
- audit trail available;
- feedback capture working;
- security and access controls accepted;
- UAT completed against the agreed dataset.

B.5 Later Phase Candidate Deliverables

Later phase candidate deliverables include:

- Google Docs integration;
- Google Drive versioning workflows;
- additional CMS integrations;
- dashboards and analytics;
- rule management UI;
- entity management UI;
- social media review;
- video/program script review;
- English support;
- multilingual support;
- Asana or workflow automation.

These are candidate deliverables, not Phase 1 commitments.

B.6 Dependency Matrix

Dependency	Needed For	Owner
Sample Arabic articles	Phase 0 benchmark	Al Jazeera
Selected editorial rules	Rule structuring and retrieval	Al Jazeera
Entity examples	Entity repository setup	Al Jazeera
Expected outputs / examples	Evaluation	Al Jazeera
Editorial reviewers	Feedback and adjudication	Al Jazeera

WordPress/CMS access	Phase 1 integration	Al Jazeera Technology Team
GCP/security standards	Environment design	Al Jazeera Technology/Security
Implementation team	Build and delivery	Delivery Team

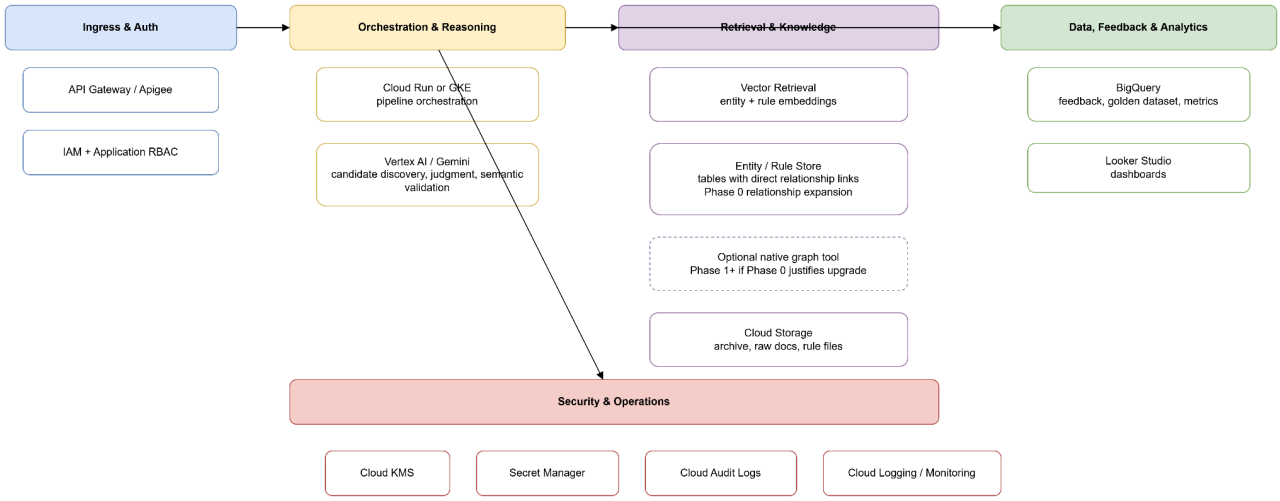
B.7 Responsibility Matrix

Area	Al Jazeera	Delivery Team
Editorial policy ownership	Accountable	Supports structuring
Rule/entity approval	Accountable	Implements workflow
Technical implementation	Reviews/approves	Accountable
Security standards	Accountable	Implements accordingly
UAT participation	Accountable	Supports and reports
Production handover	Joint	Joint

Appendix C — GCP and Integration Details

C.1 GCP Service Mapping

Figure C.1 — GCP components grouped by function:



Capability	Candidate GCP / Platform Component
LLM reasoning	Vertex AI / Gemini
Application runtime	Cloud Run or GKE
API exposure	API Gateway or Apigee, based on Al Jazeera standards
Rule/entity storage	Cloud Storage, database layer, or approved application database
Relationship expansion	Phase 0 table-based direct links between entities, rules, profiles, and related records
Optional graph upgrade	Native graph tool or graph-capable architecture if Phase 0 shows clear value
Retrieval	Vertex AI Vector Search or approved retrieval architecture
Analytics and audit	BigQuery

Dashboards	Looker Studio or approved BI layer
Logging and monitoring	Cloud Logging and Cloud Monitoring
Secrets	Secret Manager

Final service selection should be confirmed during Phase 0 and early Phase 1.

C.2 Cloud Run / GKE Considerations

Cloud Run may be suitable if the runtime can be implemented as stateless services with autoscaling and manageable execution times.

GKE may be considered if Al Jazeera requires Kubernetes-based deployment, complex networking, long-running workloads, or shared platform standards.

The proposal should not force one option before Al Jazeera's infrastructure preferences are confirmed.

C.3 Vertex AI / Gemini Usage

Gemini may be used for:

- lightweight candidate discovery for anchor-less relational rule triggers;
- judgment over curated rule/entity/risk context on the fast path;
- resolution and context clarification on the escalated path;
- semantic validation of high-risk outputs;
- rule authoring assistance in governed workflows;
- explanation support through structured proof objects.

Gemini should not be used as the source of Al Jazeera policy. Policy must come from approved rules, entity records, examples, and governed editorial knowledge.

The lite LLM candidate-discovery step should not decide that a violation exists. It should only produce structured candidate spans and risk families that drive rule retrieval and routing.

C.4 Vector Retrieval Options

Retrieval may combine:

- exact lookup;
- normalized Arabic matching;

- fuzzy matching;
- embedding/vector retrieval;
- entity alias matching;
- semantic-field matching;
- rule/entity relationship expansion;
- LLM-discovered rule-trigger risk families.

For Phase 0, relationship expansion should be implemented in the simplest reliable way. A native graph tool is not required to validate the concept. A table-based structure is sufficient, where records can carry direct links to related records.

A simple Phase 0 structure may include:

Record Type	Relationship Link Example
Entity	linked to aliases, related entities, sensitive profiles, related rule families
Rule card	linked to entities, semantic fields, examples, exceptions
Semantic field	linked to rule families and examples
Historical edit	linked to rule card, entity, risk family, and accepted/rejected outcome

The retrieval process can expand one hop from a detected entity, rule trigger, or semantic field and add those linked records to the context payload. This provides graph-like benefit without introducing a full graph platform prematurely.

Based on Phase 0 findings, Phase 1 may choose one of two options:

Option	When to Use

Continue table-based relationship expansion	If one-hop links provide enough retrieval quality and governance simplicity
Upgrade to native graph tool	If relationship depth, query complexity, rule authoring, or explainability needs exceed the table-based model

The implementation should prioritize recall first, then improve precision through payload assembly, routing, targeted retrieval on the escalated path, judgment, validation, and editor feedback.

C.5 BigQuery and Analytics

BigQuery may store:

- review events;
- feedback records;
- validation results;
- latency metrics;
- rule performance;
- acceptance/rejection trends;
- audit summaries;
- benchmark and UAT results.

Sensitive unpublished content retention should follow Al Jazeera-approved retention policies.

C.6 IAM, KMS, and Secret Manager

The solution should use:

- least-privilege service accounts;
- role-based access;
- managed encryption keys where required;
- Secret Manager for API keys and credentials;
- separate access policies for editors, language reviewers, administrators, and technical operators.

C.7 Logging, Monitoring, and Audit Logs

Logging should cover:

- request lifecycle;
- model calls, tagged by fast/escalated path;
- retrieval operations;
- validation failures;
- editor actions;
- rule/entity version usage;
- errors and exceptions;
- security-relevant events.

Audit logs should support traceability without exposing unnecessary unpublished content.

C.8 WordPress Integration

Phase 1 WordPress/CMS integration should support:

- article submission for review;
- receiving structured findings;
- rendering highlights and suggestions;
- accept/reject/edit/escalate actions;
- storing review status;
- preserving original and reviewed versions where required.

C.9 API Contracts

The API should be versioned from Phase 1 to support later integrations. Typical endpoints may include:

```
POST /v1/reviews
GET  /v1/reviews/{review_id}
POST /v1/reviews/{review_id}/feedback
GET  /v1/rules/{rule_id}
GET  /v1/entities/{entity_id}
```

Final API design should be confirmed during Phase 1 technical design.

C.10 Future Integration Model

Future integrations should call the same core API. This allows Google Docs, additional CMS platforms, dashboards, workflow tools, or custom interfaces to reuse the same reasoning core without redesign.

Appendix D — Evaluation and Benchmarking Details

D.1 Initial Benchmark Dataset

The initial Phase 0 benchmark may use 20–30 representative Arabic articles, including a mix of:

- ordinary proofreading cases;
- mechanical rule cases;
- entity naming cases;
- headline/caption examples;
- quote/source examples;
- selected sensitive wording cases.

This is an initial feasibility dataset, not final production calibration.

D.2 Golden Dataset Construction

The golden dataset should be built after Phase 0 using:

- real article samples;
- known issues;
- editor-reviewed expected outputs;
- rule/entity annotations;
- accepted and rejected examples;
- adversarial examples;
- ambiguous cases with adjudicated labels.

The golden dataset should be versioned and periodically reviewed.

D.3 Mechanical Metrics

Mechanical metrics may include:

- detection recall;
- precision;
- recommendation accuracy;

- offset accuracy;
- duplicate rate;
- false positive rate;
- consistency across reruns.

D.4 Relational Metrics

Relational metrics may include:

- entity resolution accuracy;
- context-zone accuracy;
- candidate rule-trigger detection quality;
- relationship expansion usefulness;
- correct rule retrieval;
- correct rule activation;
- severity classification quality;
- suggestion safety;
- attribution preservation;
- stance preservation;
- missed high-risk issue rate;
- editor acceptance rate;
- routing accuracy, meaning whether the router sent the case down the path it actually needed.

Relational evaluation should include anchor-less examples where no named entity, alias, or obvious banned keyword is present. These cases are necessary to test loaded framing, implicit blame, unattributed claims, attribution-strength shifts, source/voice ambiguity, and headline/caption framing.

Relationship expansion should be evaluated by checking whether direct links improved rule retrieval quality, reduced missed relevant rules, improved proof-object explainability, or introduced excessive noise.

D.5 Inter-Annotator Agreement

For relational rules, more than one valid editorial judgment may exist. Where useful, Al Jazeera may use multiple reviewers to measure agreement and define acceptable decision ranges.

The evaluation should not force false certainty where editorial judgment is genuinely required.

D.6 Latency Metrics

Latency should be measured across:

- text spine generation;
- mechanical recommendation generation;
- entity/alias/lexical candidate discovery;
- lite LLM candidate discovery;
- rule retrieval and payload assembly;
- routing;
- fast-path judgment;
- escalated-path resolution / targeted retrieval / judgment;
- deterministic validation;
- semantic validation;
- repair loop;
- total review time.

Latency should be reported by mode, article length, risk category, and path taken.

Because Al Jazeera breaking news may frequently involve conflict, political sensitivity, or attribution risk, the Phase 0 benchmark should report not only latency but also escalation rate.

D.7 Editor Acceptance Metrics

The feedback model should distinguish:

- accepted as-is;
- accepted with edit;
- rejected due to wrong rule;
- rejected due to wrong entity;
- rejected due to poor wording;
- escalated;
- ignored;
- validation failed.

D.8 Regression Testing

Regression testing should run before changes to:

- rules;
- entities;
- relationship links;
- prompts;
- lite LLM candidate-discovery logic;
- retrieval settings;
- routing policy;

- validation logic;
- model versions.

The regression report should show whether quality, safety, and latency improved or degraded.

Regression tests should include anchor-less relational cases and entity-linked cases to ensure that candidate-discovery and relationship-expansion changes do not silently reduce detection of loaded framing, implicit blame, unattributed claims, source/voice ambiguity, or entity-sensitive rules.

D.9 Release Gates

A change should not be promoted unless it passes agreed checks for:

- no critical regression;
- no high-risk false negative increase;
- acceptable latency impact;
- validation still functioning;
- audit records complete;
- responsible owner approval.

Appendix E — Editorial Governance and Responsible AI Detail

E.1 Allowed Actions

The system may:

- detect candidate issues;
- generate recommendations;
- classify decision level;
- retrieve relevant rules/entities;
- explain findings;
- validate suggestions;
- capture editor feedback;
- support rule authoring workflows.

E.2 Reviewable Actions

The following require editor review:

- sensitive descriptors;
- source/quote/voice issues;
- headline framing;
- caption framing;
- attribution changes;
- relational rule findings;
- hard warnings;
- banned usage flags;
- validation-failed suggestions.

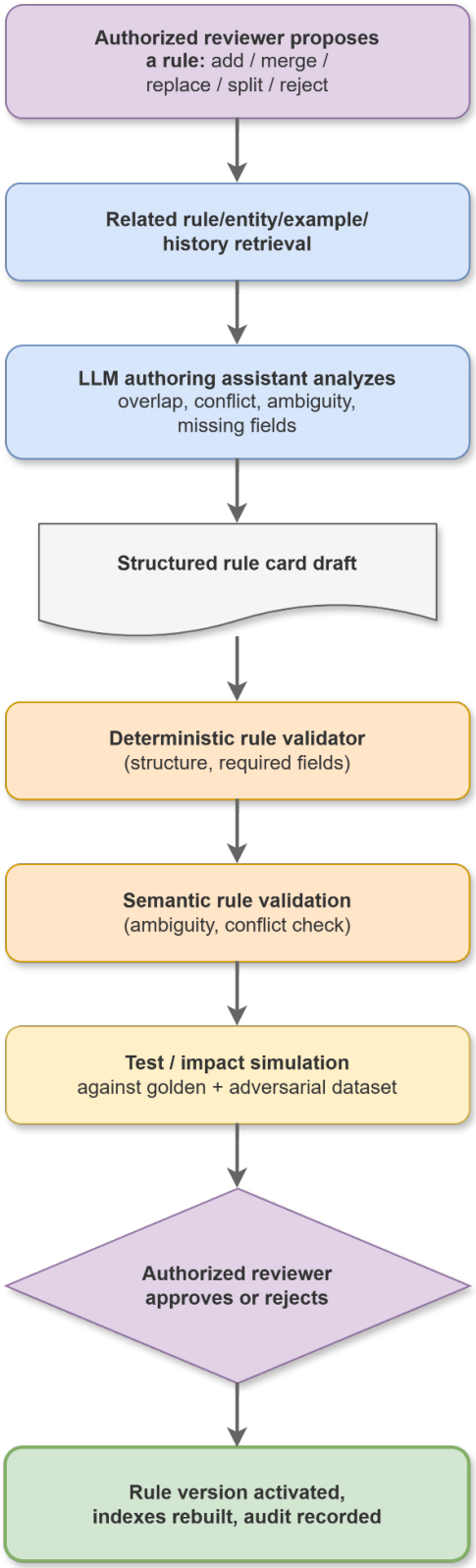
E.3 Prohibited Actions

The system must not:

- publish content;
- approve content;
- add unsupported facts;
- remove facts;
- change editorial stance;
- rewrite articles freely;
- change conflict-party descriptions without editor decision;
- remove attribution in a meaning-changing way;
- silently replace sensitive wording;
- bypass editorial hierarchy.

E.4 Rule Authoring Workflow

Figure E.4 — the full rule authoring pipeline:


Authorized reviewer proposes a rule: add / merge / replace / split / reject
Related rule/entity/example/ history retrieval
LLM authoring assistant analyzes overlap, conflict, ambiguity, missing fields
Structured rule card draft
Deterministic rule validator (structure, required fields)
Semantic rule validation (ambiguity, conflict check)
Test / impact simulation against golden + adversarial dataset
Authorized reviewer approves or rejects
Rule version activated, indexes rebuilt, audit recorded

The authoring assistant may support drafting and analysis at steps B–D, but approval at step H remains human, and activation at step I never happens without it.

E.5 Rule Versioning

Each rule should have:

- rule ID;
- version;
- effective date;
- status;
- owner;
- approval record;
- examples;
- exceptions;
- related entities;
- change history.

E.6 Entity Governance

Each entity should have:

- entity ID;
- approved name;
- aliases;
- type;
- temporal status;
- sensitive profile flags;
- related rules;
- approval record;
- version history.

E.7 Meaning and Attribution Preservation

The system should avoid suggestions that:

- add certainty;
- remove attribution;
- change source voice into Al Jazeera voice;
- add motive;
- soften or intensify claims without basis;

- introduce unsupported editorial framing.

E.8 Stance Drift Handling

Stance drift means a suggestion changes the editorial position, framing, or implied judgment of the text. The system should detect and prevent stance drift through retrieved rule constraints, semantic validation, fail-safe review, editor approval requirements, and audit logging.

E.9 Fail-Safe to Editor Review

When uncertain, the system should return a finding such as:

```
{
 "decision": "needs_editor_review",
 "reason": "The system could not validate a safe suggestion under the active rule context."
}
```

This is the correct behavior for sensitive editorial content.

Appendix F — Security and Compliance Detail

F.1 Data Protection Principles

The solution should protect unpublished editorial content through:

- approved hosting environment;
- encryption;
- access control;
- audit logging;
- retention policies;
- restricted data sharing;
- clear operational ownership.

F.2 Encryption in Transit and at Rest

The solution should use TLS for data in transit and GCP-managed or customer-managed encryption keys for data at rest, according to Al Jazeera's security standards.

F.3 Access Control

Access should be role-based and least-privilege.

Typical roles may include:

- editor;
- senior editor;
- language reviewer;
- administrator;
- technical operator;
- auditor.

Permissions should differ for reviewing content, approving rules, viewing audit logs, managing entities, and operating infrastructure.

F.4 Data Retention and Deletion

Retention policies should be defined for:

- raw submitted articles;
- AI-reviewed outputs;
- final versions;
- feedback records;
- audit logs;
- benchmark datasets;
- temporary processing data.

Deletion policies should follow Al Jazeera's legal and operational requirements.

F.5 No Unauthorized Training or Data Sharing

Unpublished content should not be used for model training or shared outside the approved environment unless explicitly approved by Al Jazeera.

The system should use approved model and cloud services only.

F.6 Audit Logging

Audit logs should include:

- user actions;
- review requests;
- generated suggestions;
- accepted/rejected/edited outputs;
- rule changes;
- entity changes;
- model/prompt/routing versions;
- validation results;
- administrative actions.

F.7 Monitoring and Alerts

Monitoring should include alerts for:

- abnormal usage;
- repeated processing failures;
- high validation failure rates;
- latency spikes;
- unauthorized access attempts;
- integration failures;
- model or retrieval errors.

F.8 Environment Isolation

Development, staging, and production environments should be separated.

Production content should not be copied into non-production environments unless permitted by Al Jazeera's data policies.

F.9 Compliance Assumptions

Compliance design assumes Al Jazeera will provide:

- cloud security requirements;
- retention requirements;
- access-control expectations;
- audit requirements;
- approved deployment region or environment;
- integration security standards.

These should be confirmed during Phase 0 and finalized during Phase 1 design.